

From Raw Detections to Real Intelligence

Trabalho Prático 1

Inês Matos Freire Santos (51563)

Maio 2026

Interação com Modelos de Larga Escala
Inteligência Artificial e Ciência de Dados



1 Introdução

O presente relatório descreve o desenvolvimento de uma solução tecnológica para o processamento e análise de eventos de visão computacional em ambiente de retalho. O desafio central, proposto na unidade curricular de Interação com Modelos de Larga Escala, consiste em transformar um fluxo massivo de deteções anónimas em inteligência de negócio acionável.

1.1 Descrição do Problema

O problema fundamental reside na natureza dos dados: sensores de visão computacional geram eventos de entrada, permanência e saída, mas perdem a "identidade" do indivíduo assim que este sai do campo de visão ou ocorre uma oclusão. O objetivo deste trabalho foi construir uma pipeline que recupere essa identidade através de um algoritmo de *stitching*, calcule métricas de comportamento e, finalmente, utilize um modelo de linguagem (*LLM*) para sintetizar recomendações estratégicas.

1.2 Dificuldades Identificadas no Dataset

Ao analisar o ficheiro `events.csv`, foram identificadas três dificuldades principais que moldaram a estratégia:

- **Volume e Densidade:** Com mais de 250.000 eventos, qualquer abordagem menos eficiente que $O(n)$ resultaria em tempos de processamento proibitivos. Neste caso, o sistema conseguiu processar cerca de 5.000 eventos por segundo, o que foi crítico para a viabilidade do projeto.
- **Ambiguidade Demográfica:** Foi verificada a presença simultânea de múltiplos indivíduos com atributos idênticos (ex: mesmo género e faixa etária). Isto exigiu uma gestão rigorosa de "perfis ativos" para evitar que as trajetórias se misturassem.
- **Fragmentação Temporal:** Muitas deteções apresentam pequenos intervalos vazios ou interrupções numa sequência que deveria ser contínua. Foi necessário definir janelas de tolerância para que uma breve saída de campo não fosse interpretada incorretamente como o fim de uma visita.

1.3 Decisão de Arquitetura: Separação Python/LLM

Uma das decisões mais estruturantes deste projeto foi a separação clara entre a lógica determinística e a lógica generativa.

1. **Componente Python (Determinística):** Optei por realizar todos os cálculos matemáticos, o *stitching* e a análise estatística de anomalias em Python puro. Esta escolha garante que as métricas sejam 100% precisas e verificáveis, eliminando o risco de o modelo "inventar" números.
2. **Componente LLM (Generativa):** O modelo llama3.1:8b (via Ollama) foi utilizado exclusivamente para a camada de interpretação e síntese. O LLM recebe os resultados processados em formato JSON e atua como um consultor de gestão, transformando dados frios em recomendações em linguagem natural.

Esta arquitetura híbrida permite-nos ter o melhor dos dois mundos: a precisão do código e a capacidade analítica da inteligência artificial.

2 Algoritmo de Stitching

O processo de reconstrução das trajetórias, designado por *stitching*, constitui a base fundamental deste trabalho, permitindo converter eventos isolados e anónimos numa visão contínua do comportamento do cliente. A lógica central do algoritmo baseia-se na premissa de que, embora os dados não possuam um identificador único de cliente, a combinação de atributos demográficos e proximidade temporal permite associar eventos com elevada confiança.

Para implementar esta associação, o algoritmo utiliza três heurísticas principais. Primeiro, é aplicada uma filtragem rigorosa por atributos, onde um evento só é associado a uma jornada ativa se o género e a faixa etária forem exatamente iguais aos registados anteriormente. Em segundo lugar, estabeleceu-se uma janela de tolerância temporal (*max gap*) de 600 segundos. Esta escolha de 10 minutos é estratégica: permite que o sistema ignore pequenas lacunas causadas por zonas cegas do sensor ou obstáculos físicos na loja, mantendo a integridade da jornada sem criar fragmentações artificiais. Por fim, o algoritmo valida a sequência lógica dos eventos, garantindo que a transição entre estados (entrada, permanência e saída) respeita a ordem cronológica real.

Em termos de eficiência, a utilização de estruturas de dados baseadas em dicionários (*hash maps*) permitiu que o algoritmo operasse com uma complexidade de $O(n)$. Esta escolha técnica foi crucial para processar o volume massivo de 250.015 eventos em aproximadamente 40 segundos num computador pessoal, cumprindo folgadoamente os requisitos de performance do enunciado.

Os resultados obtidos através do *harness* de avaliação validam estas escolhas, apresentando uma consistência de 100%, o que garante a ausência de sobreposições temporais (um cliente nunca está em dois locais ao mesmo tempo). A cobertura foi também total (100%), indicando que todos os eventos do dataset foram integrados. A métrica de completude fixou-se nos 42,86%, um valor que, embora pareça baixo, é justificável pela natureza dos dados reais, onde muitos clientes iniciam ou terminam os seus percursos em zonas não cobertas pelos sensores de entrada e saída.

Contudo, o algoritmo apresenta limitações em cenários de elevada densidade. O principal caso de falha ocorre na "colisão de atributos": se dois clientes com o mesmo perfil

demográfico cruzarem caminhos em zonas sem cobertura, o sistema pode fundir as suas trajetórias. Adicionalmente, períodos de inatividade superiores a 10 minutos resultam na divisão da jornada, um compromisso necessário para evitar que clientes diferentes sejam erradamente identificados como o mesmo indivíduo ao longo do dia.

3 Pipeline Analítico

Depois de reconstruídas as trajetórias, o sistema utiliza o módulo `analytics.py` para realizar uma análise detalhada. O objetivo aqui é transformar a lista de movimentos individuais em indicadores e métricas que ajudem a compreender como os clientes se comportam na loja. Esta etapa é fundamental, pois permite entregar à IA dados já organizados e verificados, evitando que o modelo se perca com informações irrelevantes ou "ruído".

As métricas calculadas estão divididas em três grandes grupos. Primeiro, o tráfego geral, onde foram registados 2.751 visitantes únicos e mais de 85 mil movimentos no total. Depois, foi feita a análise de cada zona individualmente, calculando o tempo médio de permanência (*dwel time*) e a taxa de paragem (*stop rate*). Esta última é muito importante, pois mostra a percentagem de pessoas que realmente pararam numa zona em comparação com todas as que apenas passaram por lá. Por último, foi criado um "funil de conversão" que segue o percurso desde a entrada até às caixas de pagamento (*Z_CK*), ajudando a perceber em que zonas a loja perde mais potenciais clientes.

Durante este processo, foram tomadas decisões sobre o que incluir no relatório final. Optei por excluir percursos muito curtos (menos de dois segundos) ou que não mostrassem movimento entre zonas. Isto foi feito para limpar o "ruído técnico", como deteções falsas ou pessoas que apenas passam pela porta sem entrar na loja.

No que toca às anomalias, embora o sistema tenha detetado mais de 1.000 desvios, decidi filtrar e manter apenas as 50 mais graves (com desvios acima de 50% da média). Assim, é garantido que o modelo de linguagem se foca apenas no que é realmente importante para a gestão do negócio.

Todos estes dados são guardados no ficheiro `metrics.json`. Foi escolhida uma estrutura organizada por categorias (*traffic_general*, *zones* e *anomalies*) para que o ficheiro seja fácil de ler tanto por nós como pela máquina. Ao organizar as métricas desta forma, o trabalho da IA é facilitado na altura de comparar o desempenho de diferentes secções. Com este formato, é assegurado que as conclusões do sistema são baseadas em números reais e precisos, como se espera.

4 Engenharia de Prompts

A inteligência do sistema depende diretamente da forma como comunicamos com o modelo de linguagem. Para este projeto, o `llama3.1:8b` foi testado com duas abordagens diferentes para perceber como a estrutura das instruções influencia a qualidade dos conselhos de negócio gerados.

4.1 Estratégia A: Zero-Shot (Sem Exemplos)

Nesta estratégia, foi enviado o ficheiro `metrics.json` ao modelo com uma instrução direta, esperando que ele soubesse interpretar os dados sozinho. O prompt pedia para o modelo

assumir o papel de analista sénior e gerar cinco *insights* em formato JSON.

O modelo teve dificuldades em entender o contexto físico da loja. Por exemplo, sugeriu a criação de "áreas de descanso" na zona Z_E2 (que é uma porta de entrada), o que bloquearia o fluxo de clientes. Além disso, verificámos uma alucinação grave: o modelo inventou que os clientes tinham idades entre os 25 e 45 anos, uma informação que não constava nos dados fornecidos.

4.2 Estratégia B: Few-Shot (Com Exemplos)

Para corrigir os erros da primeira abordagem, foi utilizado o ficheiro `few_shot.txt`. Aqui, é fornecido ao modelo exemplos de como deve ser um raciocínio de retalho, utilizando a estrutura: Observação → Implicação → Recomendação.

A melhoria foi imediata e drástica. No insight sobre a zona Z_C2, o modelo não só leu os dados, como calculou que os 2300 visitantes representavam 83% do tráfego total, sugerindo otimizações para evitar congestionamentos. Ao contrário do Zero-shot, esta estratégia não inventou dados e focou-se em zonas críticas identificadas pelo algoritmo de anomalias, como a Z_S6.

4.3 Comparação

A tabela seguinte resume a comparação entre as duas experiências:

Característica	Estratégia A (Zero-Shot)	Estratégia B (Few-Shot)
Rigor Numérico	Baixo (inventou percentagens e idades).	Elevado (usou os totais do ficheiro corretamente).
Sugestões	Genéricas e por vezes desajustadas ao layout.	Práticas e focadas em eficiência operacional.
Formato JSON	Correto, mas com conteúdo vago.	Perfeito e com dados cruzados.

Tabela 1: Análise comparativa das estratégias de Prompting.

Concluimos que a técnica de *Few-shot* é obrigatória para este tipo de sistema. Sem os exemplos, o modelo tende a "preencher os espaços vazios" com informações inventadas. Com os exemplos, o Llama transformou-se numa ferramenta útil que realmente ajuda um gestor a tomar decisões sobre o fluxo da sua loja.

5 Avaliação e Resultados

A avaliação final do sistema foi realizada utilizando o *harness* de testes (`evaluate.py`), que permite medir a qualidade da reconstrução das trajetórias e a robustez da pipeline. Os resultados obtidos no dataset de treino demonstram que as escolhas algorítmicas foram eficazes, embora revelem também as dificuldades inerentes à natureza dos dados reais.

5.1 Resultados do Harness de Avaliação

O sistema apresentou os seguintes indicadores de desempenho:

- **Consistência (100%):** Este é o resultado mais crítico. Significa que o algoritmo de *stitching* impediu com sucesso que qualquer cliente estivesse em duas zonas ao mesmo tempo. A lógica de "perfis ativos" garantiu que as jornadas fossem logicamente impossíveis de se sobreporem.
- **Cobertura (100%):** Todos os 250 015 eventos do ficheiro original foram processados e atribuídos a uma visita. Isto prova que o sistema é resiliente e não descarta dados por erro de leitura ou formatação.
- **Completude (42,86%):** Este valor indica a percentagem de trajetórias que começam numa entrada e terminam no checkout. Embora pareça um valor baixo, este é justificável: num ambiente de retalho real, muitos clientes já estão na loja quando o sensor liga ou abandonam a loja sem passar pelo funil principal de saída. O sistema detetou estas jornadas "incompletas" mas manteve a sua integridade estatística.

5.2 Análise de Falhas e Limitações Honestas

Apesar dos bons resultados numéricos, o sistema possui limitações que devem ser reconhecidas. A principal falha reside na dependência exclusiva de metadados demográficos. Caso dois clientes com o mesmo perfil (ex: dois "Homens Adultos") se cruzem numa zona sem cobertura de sensores, o sistema pode confundir as suas trajetórias, uma vez que não possui reconhecimento facial para os distinguir.

Outra limitação importante prende-se com a inteligência artificial. Como demonstrado na comparação entre as estratégias de *prompting*, o modelo `llama3.1:8b` é propenso a alucinações se não for devidamente guiado. No teste *Zero-shot*, o modelo chegou a inventar dados demográficos e sugeriu alterações de layout fisicamente impossíveis. Embora a estratégia *Few-shot* tenha mitigado quase totalmente estes erros, o sistema continua a estar limitado pela capacidade de raciocínio do modelo local, que pode ter dificuldade em correlacionar anomalias muito complexas entre diferentes dias da semana.

5.3 Testes de Robustez com Anomalias

Para testar a capacidade de deteção do sistema, foi utilizado o dataset `test_anomalies.csv`, onde foi injetada uma quebra artificial de 80% no tráfego da zona Z_FAT. O pipeline de *Analytics* identificou corretamente o desvio estatístico e o módulo de *Insights* gerou um alerta prioritário.

6 Conclusão

O desenvolvimento deste projeto permitiu transformar com sucesso um fluxo massivo de eventos anónimos numa ferramenta de inteligência estratégica para o retalho. Ao longo deste trabalho, foi possível validar que a eficácia de um modelo de linguagem (LLM) depende da qualidade e da organização dos dados que lhe são fornecidos na fase de pré-processamento.

6.1 O que funcionou

O algoritmo de *stitching* revelou-se extremamente eficiente. A escolha de uma complexidade $O(n)$ permitiu processar mais de 250 mil eventos em menos de um minuto, garantindo

que o sistema é escalável para lojas com ainda mais tráfego. No campo da inteligência artificial, a estratégia de *Few-shot prompting* foi o maior sucesso, eliminando as alucinações de dados e garantindo que as recomendações fossem práticas, como a gestão de filas nas caixas ou a análise de anomalias em zonas específicas.

6.2 O que não funcionou e Limitações

Nem todos os aspetos do sistema são perfeitos. A maior dificuldade encontrada foi a dependência de metadados demográficos genéricos. Sem uma forma de identificação única (como biometria), o sistema terá sempre uma margem de erro quando pessoas com o mesmo perfil se cruzam em zonas mortas dos sensores. Além disso, a estratégia *Zero-shot* provou ser inadequada para este contexto, uma vez que o modelo tendia a inventar informações para preencher a falta de contexto sobre o funcionamento físico da loja.

6.3 O que faria de diferente

Se voltasse a iniciar o projeto, a principal alteração seria a integração dos dados de *walk_seconds* presentes no ficheiro `zones.json` diretamente na lógica de associação do *stitcher*. Isto permitiria descartar trajetórias que são fisicamente impossíveis (ex: um cliente aparecer numa zona distante em apenas um segundo).

Outra melhoria seria a criação de um sistema de "memória" a longo prazo (RAG - *Retrieval-Augmented Generation*). Em vez de analisar apenas uma semana isolada, o sistema poderia comparar os dados atuais com o histórico da loja, permitindo à IA detetar se uma anomalia é um evento único ou uma tendência negativa que se está a repetir.

Em suma, este projeto demonstrou que a "Inteligência Real" mencionada no enunciado nasce da combinação entre algoritmos clássicos robustos e a flexibilidade interpretativa dos modelos de linguagem modernos.